# Data Management Engineering Assignment 1

Group 3

Jaime Vergara Lidia Stankova Virag Bognar Rares Popa
Zhili Yang Wu
Technical University Delft

October 11, 2026

## 1 Introduction

This report describes the Part I design of our data pipeline for three quantum error-correction (QEC) sources: simulated syndrome sequences (`qec_syndromes`), Google surface-code hardware experiments (`google_qec`), and QASMBench circuits (`qasmbench`). The pipeline verifies the supplied files without changing them, checks and cleans them into source-specific Parquet tables (Silver), loads them into a relational PostgreSQL model (Gold), and exports the two required ML tables from Gold only. Every exported example can be traced back to the exact source bytes it came from. Part II results are reported separately in `results/part2/report.md`.

## 2 Architecture

Our pipeline implements a Bronze–Silver–Gold architecture followed by an explicit ML hand-off. The three supplied QEC sources are first preserved in Bronze, then parsed and validated into source-specific Silver Parquet tables. These tables are subsequently loaded into a PostgreSQL Gold database, where the data is connected through a relational model and made available for analysis and ML export.

The **Bronze layer** contains the original syndrome, Google QEC, and QASMBench source objects. We keep these files unchanged and verify their SHA-256 hashes and archive members before processing them. This provides a fixed reference against which all downstream results can be traced.

In **Silver**, we parse the different source formats into six Parquet tables. The syndrome data remains at its aggregate observation grain, while each Google shot is represented individually and aligned with its measurement, detector, observable-flip, and decoder-prediction data. QASMBench is represented through circuits, stabilizer checks, and conditional corrections. Each row receives a stable `source_record_id`, with detailed provenance stored separately in `source_trace.parquet`. Invalid records are excluded from Silver and recorded in `data_issues.parquet`.

The **Gold layer** is implemented in PostgreSQL. We use separate relations for experiments, syndrome patterns and observations, Google shots, decoders and predictions, circuits, stabilizer checks, qubits, and conditional corrections. Detector information is additionally exposed through an experiment detector-summary view. Primary keys, foreign keys, checks, and indexes enforce the main relational rules. The three sources are kept connected at the conceptual QEC level without creating unsupported experiment matches; in particular, QASMBench has no reliable identifier linking its circuits to the experimental datasets.

Finally, the planned **ML layer** uses SQL queries over Gold to construct the two required model-input Parquet tables. This keeps the downstream models independent of the original source files while retaining a path from each example back through Gold and Silver to its Bronze source. The pipeline also stores run metadata, row counts, quality issues, and trace information under `results/part1/`.

# 3 Bronze: Source Verification

Bronze holds the three course archives exactly as released. The release manifest (`quantum-data-core`, bundle version 3, 5 August 2026) lists each archive with its size and SHA-256 hash. `make bootstrap` copies the archives from a read-only mount of the course data into the MinIO bucket `quantum-lake`, under `bronze/source=<name>/`. No pipeline stage writes to Bronze or extracts archives to disk: later stages read the archive bytes from MinIO into memory.

Table 1: Summary of Bronze tier datasets.

| Source | Bronze Object | Bytes | Members |
|---|---|---|---|
| `qec_syndromes` | `syndromes_dataset.zip` | 358 017 | 8 |
| `google_qec` | `google-surface-code-curated.zip` | 14 638 673 | 76 |
| `qasmbench` | `qasmbench-qec.zip` | 144 172 | 19 |

## 3.1 Verification (Stage 1, `register_sources`)

The first stage of `make run` compares the Bronze objects in MinIO with the release manifest. It checks:

- **Presence:** every mandatory manifest object exists in Bronze.
- **Size:** the object size equals the manifest's byte count.
- **Content:** the SHA-256 hash of the object's bytes equals the manifest hash.
- **Safe archive members:** no ZIP member has an absolute path, a `..` path component or a backslash, any of which could write outside the target folder if the archive were extracted.

- **Unexpected objects:** objects in Bronze that the manifest does not list are reported as warnings.

A missing object, a size or hash mismatch, or an unsafe member path stops the run before any Silver data is written. The pipeline therefore never builds tables from altered or incomplete sources. Unexpected extra objects only produce a warning, because they do not change the content of the release. On the current release all three objects are verified, with no unexpected objects and no unsafe members.

### 3.2 Run Evidence and Identifiers

The verified hashes are written to `results/part1/run.json` (`input_hashes`) together with the code revision, start and end times and all output counts. A result can therefore be tied to the exact inputs and code that produced it.

Every record read from Bronze gets a stable `source_record_id` built from the source name, the archive member and the record's position. For example:

- `qec_syndromes:d-3_pfr-0.005000_nb-10M.csv:row:0` is the first data row of one syndrome file;
- `google_qec:surface_code_bX_d3_r25_center_3_5:shot:1` is a shot of one Google experiment.

The identifier depends only on the source content, never on run time or insertion order, so it is the same on every run. `results/part1/source_trace.parquet` maps each identifier to its Bronze object, archive member, record locator and input hash: 2,075,623 trace rows in the current run.

### 3.3 Repeated Runs

Stage 1 only reads Bronze, so a second run has nothing to duplicate. Because identifiers are derived from content, a rerun produces the same Silver identifiers, and the Gold load can recognise records that already exist. Two consecutive runs produce identical Silver, Gold and ML outputs; the repeated-run tests check this.

## 4 Source Discovery

Before designing Silver and Gold we inspected every archive member and its documentation to establish what one record means, what identifies it, and which values are nested, weighted, repeated or packed.

### 4.1 QEC Syndromes (Simulation)

The archive contains seven CSV files and a `README.txt`. Each file is one simulated experiment: a distance-3 surface code at one physical fault rate (from $10^{-5}$ to $10^{-2}$), with 10 million samples. The fault rate and sample count are encoded only in the file name (`d-3_pfr-0.005000_nb-10M.csv`).

- **Row meaning:** one row is one *distinct* combination of logical-error label and syndrome sequence. The `quantity` column counts how many of the 10 million samples had that combination. The rows are therefore aggregated and weighted: each file has between 68 and 49,676 rows, but the quantities always sum to 10,000,000 (70 million samples in total). Unweighted row counts would misrepresent the physics; all analyses and models use `quantity` as the weight.
- **Nested structure:** the syndrome is a text-encoded tuple of 4 rounds $\times$ 4 checks, for example `((0, 0, 0, 0), (0, 0, 0, 0), ...)`. Order matters: the position gives the round and the check.
- **Same syndrome, both labels:** 25,159 (file, syndrome) pairs occur with label 0 and with label 1. This is valid data, not a duplicate, because a syndrome does not determine whether a logical error occurred. Both rows are kept.
- **Documentation mismatch:** the README calls the label column `label`, but the CSV header is `labels`. We follow the header and record the difference as a quality observation.
- **Repetition across files:** the same syndrome pattern appears in several files; there are 31,941 distinct patterns across the 75,598 rows. Gold stores each pattern once (`syndrome_pattern`).

## 4.2 Google Surface-code Experiments (Hardware)

The archive holds five experiment directories and a `README.txt`. The directory name encodes the experiment: `surface_code_bX_d3_r25_center_3_5` means X basis, distance 3, 25 rounds, centre data qubit at row 3, column 5. Four experiments are distance 3 at different chip locations; one is distance 5. Each directory has a `properties.yml` (50,000 shots; 200 detectors at distance 3 and 600 at distance 5) and these data files:

- `measurements.b8`, `detection_events.b8`, `sweep.b8`: one bit-packed record per shot;
- `obs_flips_actual.01`: the actual logical observable flip, one line per shot;
- four `obs_flips_predicted_by_<decoder>.01` files: the predictions of belief matching, correlated matching, PyMatching and tensor-network contraction;
- circuit and error-model files (`.stim`, `.dem`) and a layout image, which we keep in Bronze but do not model.

**Row meaning and identifiers.** One record is one shot. The files contain no shot identifier: record $n$ of every file in a directory belongs to shot $n$. The stable identifier is therefore the experiment name plus the shot index, and all files must have exactly the declared number of shots for this alignment to be valid. That gives 250,000 shots and 1,000,000 decoder predictions.

**Packed values.** `.b8` files store each shot in $\lceil n/8 \rceil$ bytes, little-endian within each byte, with unused padding bits set to zero. The 200 detector bits of a distance-3 shot therefore take 25 bytes. A wrong bit order would silently scramble the detectors, so the order and the padding are checked and tested.

### 4.3 QASMBench (Circuits)

The archive has 19 members: six `.qasm` circuits (three small error-correction benchmarks, each as a source and a transpiled variant), a README and an image per benchmark, the gate library `qelib1.inc`, and `LICENSE`/`NOTICE`. We parse each circuit into three row types:

- a **circuit** (qubit count, measurement count, operation and two-qubit-gate counts): 6 rows;
- a **stabilizer (parity) check**, identified by an ancilla qubit that receives CNOTs from data qubits and is then measured: 8 rows;
- a **conditional correction**, an `if(c==v)` gate applied for a given syndrome value: 6 rows.

Comparing the circuits with their own documentation showed that two circuit files use the gate `sx`, which is defined neither in the file nor in `qelib1.inc`; that two benchmark READMEs report gate counts that differ from the parsed circuit; and that four parity checks have no separate ancilla register, so they are inferred from the CNOT pattern. All are recorded as warnings or notes; no circuit is dropped.

### 4.4 Relationships Between Sources

The three sources share vocabulary (distance, rounds, checks/detectors, logical error) but no identifiers. The simulated syndromes, the Google hardware shots and the QASMBench circuits are different experiments, so we do not invent cross-source keys. The evidence for rejecting the QASMBench link is given in the Gold section.

## 5 Silver: data cleaning and quality

### 5.1 Cleaning Rules

Table 2 shows an overview of all the cleaning rules implemented to produce the silver tables.

Table 2: Silver data quality checks; all 24 invariants pass on the supplied release with zero fatal failures.

| Rule Identifier | Source | Invariant Checked | Checked | Failed |
|---|---|---|---|---|
| RULE_SYN_FILENAME_PFR | qec_syndromes | physical fault rate parses from the file name | 7 | 0 |
| RULE_SYN_EMPTY_FILE | qec_syndromes | CSV file has a header row | 7 | 0 |
| RULE_SYN_HEADER_DOCUMENTED | qec_syndromes | CSV header compared with the columns documented in README.txt | 7 | 0 |
| RULE_SYN_HEADER | qec_syndromes | header is labels,syndromes,quantity (or the documented label,...) | 7 | 0 |
| RULE_SYN_ROW_LEN | qec_syndromes | row has exactly 3 fields | 75 598 | 0 |
| RULE_SYN_LABEL_DOMAIN | qec_syndromes | label is 0 or 1 | 75 598 | 0 |
| RULE_SYN_SHAPE_DOMAIN | qec_syndromes | syndrome is 4 rounds x 4 checks of binary values | 75 598 | 0 |
| RULE_SYN_QUANTITY_POSITIVE | qec_syndromes | quantity is an integer greater than zero | 75 598 | 0 |
| RULE_SYN_WEIGHTED_TOTAL | qec_syndromes | accepted quantities of a file sum to the nb-¡N¿ count in its name | 7 | 0 |
| RULE_SYN_SAME_SYNDROME_BOTH_LABELS | qec_syndromes | a syndrome occurring with both labels in one file is valid and kept | 50 439 | 0 |
| RULE_EXP_PROPERTIES | google_qec | properties.yml parses with valid basis, distance, rounds and shots | 5 | 0 |
| RULE_GOOGLE_NAME_PROPERTIES | google_qec | directory name agrees with properties.yml | 5 | 0 |
| RULE_GOOGLE_MISSING_COMPANION | google_qec | every required companion file is present | 5 | 0 |
| RULE_GOOGLE_01_BINARY | google_qec | every .01 line is 0 or 1 | 25 | 0 |
| RULE_GOOGLE_COMPANION_STRIDE_MISMATCH | google_qec | b8 lengths equal shots x ceil(bits/8) and .01 line counts equal shots | 5 | 0 |
| RULE_GOOGLE_PADDING_BITS | google_qec | unused padding bits of every b8 record are zero | 250 000 | 0 |
| RULE_QASM_ARCHIVE_UNAVAILABLE | qasmbench | QASMBench archive can be read | 1 | 0 |
| RULE_QASM_CIRCUIT_PARSE | qasmbench | circuit registers and operation counts parse | 6 | 0 |
| RULE_QASM_STABILIZER_PARSE | qasmbench | parity checks parse | 6 | 0 |
| RULE_QASM_CORRECTION_PARSE | qasmbench | conditional corrections parse | 6 | 0 |
| RULE_QASM_UNDECLARED_GATE | qasmbench | every gate used is defined in file or qelib1.inc (2 warnings kept) | 6 | 0 |
| RULE_QASM_README_METRICS | qasmbench | README gate counts equal parsed source circuit (2 warnings kept) | 3 | 0 |
| RULE_QASM_INFERRED_PARITY_CHECK | qasmbench | parity check is explicit vs inferred from data CNOTs (4 notes kept) | 8 | 0 |
| RULE_QASM_NON_CIRCUIT_MEMBER | qasmbench | archive member is a .qasm circuit (13 non-circuits skipped) | 19 | 0 |

### 5.1.1 Data quality findings.

A total of 21 non-fatal data quality findings were detected during Silver preparation and preserved in results/part1/data_issues.parquet (Table 3). No rows were silently dropped; non-circuit files were safely skipped, while circuits with undeclared gates or documentation mismatches were retained with audit flags.

Table 3: Data quality issues preserved in `results/part1/data_issues.parquet`.

| Rule Identifier | Severity | Count | Action | Reason |
|---|---|---|---|---|
| RULE_QASM_NON_CIRCUIT_MEMBER | info | 13 | skipped | Non-circuit members in QASMBench archive (docs, images, licenses) |
| RULE_QASM_INFERRED_PARITY_CHECK | info | 4 | kept | Parity check inferred from CNOTs onto data register (no separate ancilla) |
| RULE_QASM_UNDECLARED_GATE | warning | 2 | kept | Gate 'sx' used without definition in file or qelib1.inc |
| RULE_QASM_README_METRICS | warning | 2 | kept | README gate counts disagree with parsed OpenQASM circuit AST |
| **Total issues** | | **21** | | *All 21 preserved in data_issues.parquet* |

### 5.1.2 Syndrome Source Rules:

The QEC syndrome cleaning rules check that the simulation data is readable, consistently structured, and valid for analysis. These also flag documentation differences and distinguish formatting errors from valid syndrome patterns. Invalid data are recorded as an issue.

The rules are as follows:

- **Filename fault rate** (RULE_SYN_FILENAME_PFR): Reads the physical fault rate from the given source CSV filename. If it is unable to parse this value, the file is excluded.
- **CSV header present** (RULE_SYN_EMPTY_FILE): Checks that the CSV is not empty and has a header row. A file without a header is excluded.
- **Header documentation comparison** (RULE_SYN_HEADER_DOCUMENTED): Compares the actual source CSV header with the columns listed in the archive's README. A mismatch is recorded as an observation, rather than treated as invalid data.
- **Recognized header** (RULE_SYN_HEADER): Checks that the header matches the expected columns. An unexpected header is logged as a warning. The parser reads row values by position.
- **Row field count** (RULE_SYN_ROW_LEN): Requires exactly three fields per data row. Otherwise, the row is excluded.
- **Label domain** (RULE_SYN_LABEL_DOMAIN): Requires the label to be either 0 or 1. Invalid labels exclude the row.
- **Syndrome shape and values** (RULE_SYN_SHAPE_DOMAIN): Requires four rounds of four checks, with all 16 values binary. Invalid syndrome rows are excluded.
- **Quantity** (RULE_SYN_QUANTITY_POSITIVE): Requires quantity to be an integer greater than zero. Invalid rows are excluded. Valid quantities are retained as weights, and the pipeline does not expand them into repeated observations.

- **Weighted total** (`RULE_SYN_WEIGHTED_TOTAL`): Adds the accepted rows' quantities per source file and compares the sum with the observation count stated in the filename. A mismatch is logged as an error, it does not silently change or remove accepted rows.
- **Same syndrome, both labels** (`RULE_SYN_SAME_SYNDROME_BOTH_LABELS`): Checks for syndrome patterns that occur with both labels within a file. This is treated as a valid observation, and both rows are kept.

#### 5.1.3 Google Source Rules:

The Google QEC cleaning rules check if an experiment's metadata align into correct shot records. Overall, these checks protect against invalid experiment metadata, missing or misaligned source files, and malformed shot values.

The rules are as follows:

- **Experiment properties** (`RULE_EXP_PROPERTIES`): Checks that an experiment's properties can be parsed and contains valid key details, including basis, distance, rounds, and shot count.
- **Directory-metadata agreement** (`RULE_GOOGLE_NAME_PROPERTIES`): Compares experiment details encoded in the directory name with the experiment's property metadata. A mismatch excludes the experiment and its shots.
- **Required companion files** (`RULE_GOOGLE_MISSING_COMPANION`): Ensures each experiment has the measurement, detector-event, actual-outcome, and decoder-prediction files it needs. The sweep file is required when the metadata says the experiment has sweep bits. Missing required files stop the run.
- **Binary outcome and prediction values** (`RULE_GOOGLE_01_BINARY`): Checks that values in the .01 actual-outcome and decoder-prediction files are binary (0 or 1).
- **Companion-file alignment** (`RULE_GOOGLE_COMPANION_STRIDE_MISMATCH`): Checks that packed `b8` file sizes and `.01` line counts match with the declared number of shots and bits per shot. A mismatch excludes the experiment so values from different shots aren't accidentally combined.
- **Packed-bit padding** (`RULE_GOOGLE_PADDING_BITS`): Ensures unused bits at the end of each packed record are zero. A shot with nonzero padding is excluded.

#### 5.1.4 QASMBench Source Rules:

The QASMBench cleaning rules that have been implemented mainly involve parseability and structural checks. If parsing a QASM file produces an error, it is excluded from the output of the current table that is being parsed and an issue is recorded.

The rules are as follows:

- **Archive availability** (`RULE_QASM_ARCHIVE_UNAVAILABLE`): Checks that the supplied QASMBench archive exists and can be opened as a ZIP file. If it cannot be read, QASMBench processing cannot continue.
- **Circuit parsing** (`RULE_QASM_CIRCUIT_PARSE`): Checks whether each .qasm file can be parsed to extract register declarations, qubit count, measurement count, and gate-operation counts. Register-wide operations are counted by their expanded applications. A parsing error rejects that circuit and is recorded as an issue.
- **Stabilizer-check parsing** (`RULE_QASM_STABILIZER_PARSE`): Looks for data-to-ancilla `CNOT` (`cx`) interactions and a measurement that maps the ancilla result to a classical syndrome bit. Qualifying checks are recorded with their ancilla, data qubits, and syndrome bit. A parsing error rejects that file's check records and is recorded as an issue.
- **Conditional-correction parsing** (`RULE_QASM_CORRECTION_PARSE`): Looks for supported conditional statements such as `if (register == integer) gate target;` and extracts the condition, gate, and target qubit. A parsing error rejects that file's correction records and is recorded as an issue. Conditional statements that don't match the supported pattern may be skipped.
- **Skip non-circuit members** (`RULE_QASM_NON_CIRCUIT_MEMBER`): Notes archive members that are not .qasm circuit files, including READMEs, images, or the `qelib1.inc` gate library. These are skipped as circuits, but the gate library can still be used to check whether gates are declared.
- **Inferred parity checks** (`RULE_QASM_INFERRED_PARITY_CHECK`): Flags a parsed parity check when its ancilla and data qubits are in the same register instead of using a separate ancilla register. The check is kept with an information note.
- **Undeclared gate** (`RULE_QASM_UNDECLARED_GATE`): Warns when a circuit uses a gate that is not declared in that circuit, in an included `qelib1.inc`, or as a recognized OpenQASM built-in. The circuit is still kept, the warning indicates that the gate's definition may be missing or unavailable to the parser.
- **README metrics** (`RULE_QASM_README_METRICS`): Compares metrics listed in the benchmark README with counts parsed from the source circuit. These metrics include qubit count, two-qubit gate count, and total gate-operation. Differences are recorded as warnings, but the circuit is kept.

These checks ensure that the implemented parser is capable of processing the relevant data. However, it is important to note that these checks do not prove that the circuit is scientifically or semantically correct.

## 5.2 Traceability

Every Silver row carries a deterministic surrogate key `source_record_id`. Full lineage is decoupled from Silver tables and stored in `results/part1/source_trace.parquet`.

**Deterministic identifiers.** Identifiers are built deterministically from the source name, archive member, and position:

- Syndromes: `qec_syndromes:<member>:row:<idx>`
- Google experiments: `google_qec:<exp_dir>:properties.yml`
- Google shots: `google_qec:<exp_dir>:shot:<shot_idx>`
- QASMBench circuits: `qasmbench:<path>:circuit`
- QASMBench stabilizer checks: `qasmbench:<path>:check:<check_num>`
- QASMBench conditional corrections: `qasmbench:<path>:statement:<statement_idx>`

**Multi-member companion tracing.** A single physical shot in `google_qec/shot.parquet` is assembled from 8 companion files. In `source_trace.parquet`, that shot expands into 8 separate rows sharing the same `source_record_id`, each pointing to its specific companion member and record locator (Table 4).

Table 4: Multi-member trace expansion for shot 0 in `surface_code_bX_d3_r25_center_3_5`.

| Shared `source_record_id` | Archive Member Path | Record Locator | Valid |
|---|---|---|---|
| `...:shot:0` | `surface_code_bX_d3_r25_center_3_5/detection_events.b8` | `shot:0` (offset 0) | `yes` |
| `...:shot:0` | `surface_code_bX_d3_r25_center_3_5/measurements.b8` | `shot:0` (offset 0) | `yes` |
| `...:shot:0` | `surface_code_bX_d3_r25_center_3_5/sweep.b8` | `shot:0` (offset 0) | `yes` |
| `...:shot:0` | `surface_code_bX_d3_r25_center_3_5/obs_flips_actual.01` | `line:0` | `yes` |
| `...:shot:0` | `.../obs_flips_predicted_by_belief_matching.01` | `line:0` | `yes` |
| `...:shot:0` | `.../obs_flips_predicted_by_correlated_matching.01` | `line:0` | `yes` |
| `...:shot:0` | `.../obs_flips_predicted_by_pymatching.01` | `line:0` | `yes` |
| `...:shot:0` | `.../obs_flips_predicted_by_tensor_network_contraction.01` | `line:0` | `yes` |

**Volume accounting.** The 2 075 623 rows in `source_trace.parquet` reconcile exactly:

$$N_{\text{trace}} = \underbrace{75\,598}_{\text{syndromes}} + \underbrace{5}_{\text{experiments}} + \underbrace{(250\,000 \times 8)}_{\text{2 000 000 Google companions}} + \underbrace{20}_{\text{QASMBench}} = 2\,075\,623 \text{ rows} \tag{1}$$

**Reconciliation and audit trail.** Table 5 shows end-to-end row conservation from `row_counts.json`. For every table, rows read equal rows accepted and loaded with 0 rejected. Running Part I twice consecutively yields identical hashes and record counts with zero duplicates.

Table 5: End-to-end row reconciliation audit trail from `row_counts.json`.

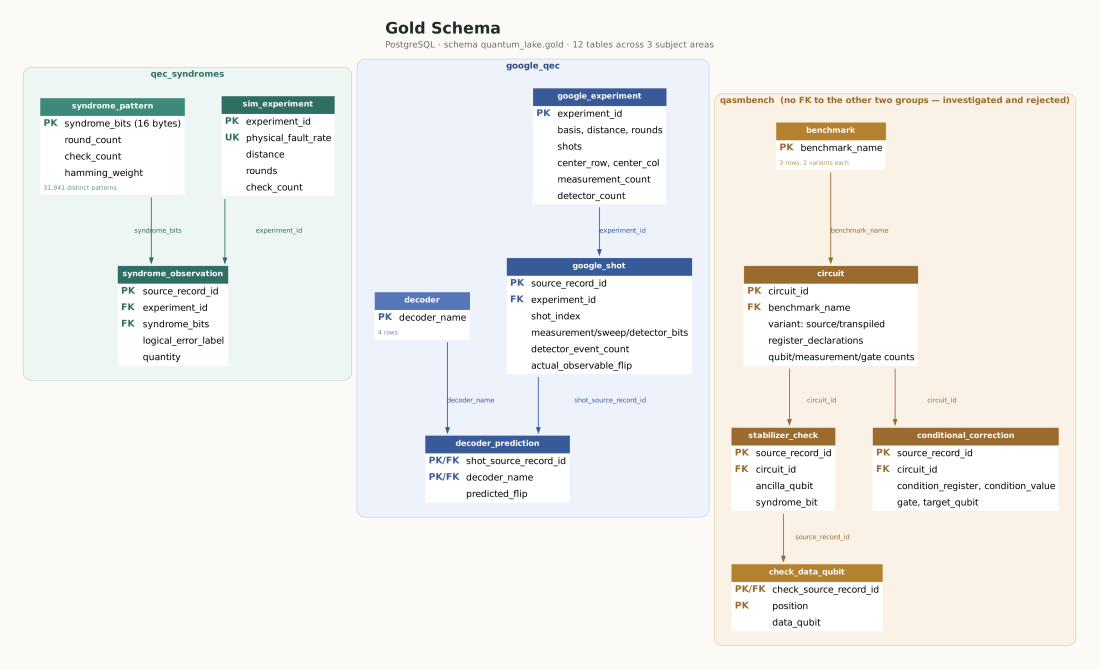
| Target Table | Read | Accepted | Rejected | Loaded | Unit |
|---|---|---|---|---|---|
| `results/part1/data_issues.parquet` | 21 | 21 | 0 | 21 | issue finding |
| `results/part1/source_trace.parquet` | 2 075 623 | 2 075 623 | 0 | 2 075 623 | trace row |
| `silver/google_qec/experiment.parquet` | 5 | 5 | 0 | 5 | experiment properties file |
| `silver/google_qec/shot.parquet` | 250 000 | 250 000 | 0 | 250 000 | declared shot |
| `silver/qasmbench/circuit.parquet` | 6 | 6 | 0 | 6 | circuit row |
| `silver/qasmbench/conditional_correction.parquet` | 6 | 6 | 0 | 6 | conditional correction row |
| `silver/qasmbench/stabilizer_check.parquet` | 8 | 8 | 0 | 8 | stabilizer check row |
| `silver/qec_syndromes/syndrome_observation.parquet` | 75 598 | 75 598 | 0 | 75 598 | CSV data row |

# 6 Gold

## 6.1 Schema

After the Silver stage, the 6 parquet tables were split into 12 tables. The gold schema can be seen in Figure 1.

Figure 1: Gold schema



### 6.1.1 QEC_syndromes

The **syndrome_observation** table got broken down into 3 separate tables: sim_experiment, syndrome_pattern and syndrome_observation. The initial Silver table contained repeated syndrome_bits + row_count + check_count on every row, and from 75598 rows we extracted 31941 distinct patterns, some of which show up on more than one fault-rate experiment. The syndrome_pattern table was created and contained the syndrome_bits as a primary key, together with the round_count, check_count and hamming_weight. The table sim_experiment contains the fields distance, rounds, check_count and physical_fault_rate as a unique key and experiment_id as primary key. The last table syndrome_observation contains just the observations of the experiment, including the fields logical_error_label, quantity, the primary key source_record_id and it is referencing the pattern table through the foreign key syndrome_bits, and the experiment table through the foreign key experiment_id.

### 6.1.2 QASMBench

The **circuit** table got broken down into 2 tables: circuit and benchmark. 6 circuit rows collapsed to 3 distinct benchmark names, each compiled twice. Benchmark contains benchmark_name as primary key. And circuit contains primary key circuit_id, and foreign key benchmark_name which references the benchmark table, and variant, register_declarations, and qubit/measurement/gate_counts.

The **stabilizer_check** got broken down into 2 tables: stabilizer_check and check_data_qubit. In the Silver stage, the initial table was storing data_qubits as a text array which are not queryable with joins, so we made it a proper table with position preserving the original order. The stabilizer_check contains primary key source_record_id and foreign key circuit_id which references the circuit table, and ancilla_bit and syndrome_bit. The check_data_qubit contains primary/foreign key check_source_record_id which references the stabilizer_check table, primary key position and data_qubit.

The **conditional_correction** remained the same as in the Silver table. It contains the primary key source_record_id, foreign key circuit_id which references the circuit table, condition_register, condition_value and gate, target_qubit.

### 6.1.3 Google_qec

The **shot** table was broken down into 3 tables: google_shot, decoder and decoder_prediction. The silver table contains 4 columns for each decoder, which means that adding an extra decoder would mean adding more columns which is impractical. The decoder table contains a primary key and that is the decoder_name. Then we have the decoder_prediction table which contains primary/foreign key shot_source_record_id and primary/foreign key decoder_name which references the decoder table, and predicted_flip. The google_shot table contains the information about shot_index, measurement/sweep/detector_bits, detector_event_count and actual_observable_flip. Decoder_prediction also references Google_shot through the primary key source_record_id.

The last table, **google_experiment** remains the same and it the rest of the information about the shot experiment like basis, distance, rounds, shots, center_row, cen-

ter_col, measurement_col and detector_col. It also references the google_shot table through the primary key experiment_id.

The schema totals to 440.27 MB, it is measured as a part of the pipeline, and it is output in the results subfolder in gold_storage_measurement.json.

We also looked at any possible relations within the QASMBench and the Google_qec and QEC_syndromes, since all 3 contained similar fields (distance, syndrome bits, checks). There were no row-level joins existent, so there are no foreign keys from circuit to google_experiment or sim_experiment. In the q4_rejected_relationship.sql we do this by first finding any similar id's between the already existent tables, and see if the qubit_count is equal to the qubits of a google patch (2*d$\hat{2}$ - 1). This checks the similarity of QASMBench circuit declared count_bit and google's qubit count.

## 6.2 Analysis

The following questions have been answered in the analysis section of the results subfolder.

### 6.2.1 How do weighted syndrome frequency and logical-error labels change with physical fault rate?

As fault rate rises from 1e-05 to 0.01, mass shifts away from the trivial (0-fired-bit) syndrome, its weighted share falls from 99.87% to 28.40% of shots, and spreads out across higher hamming weights. At 1e-05 fault rate, 1–4 fired bits together account for over 65% of weighted shots, and patterns with up to 12 fired bits appear, as opposed to a max of 4 at the lowest fault rate of 0.01. The clearest association, though, is a parity effect that's invisible in the per-fault-rate aggregate: odd hamming weights carry roughly a 50% logical-error rate at every fault rate, while even hamming weights (other than 0) carry a much lower rate that grows with fault rate. This is consistent across all 7 files. Since the weighted overall error rate is really a mix of near constant 50% rate on odd-weight syndromes and even-weight syndromes, the parity of the syndrome, is a strong association with the label, and it's the shifting weighted frequency toward higher hamming weights, more than any change in the per-weight error rates themselves, that drives most of the overall rate increase. This describes an association in the data, not a causal mechanism.

### 6.2.2 How do the supplied decoder logical-error rates compare by code distance and distance-three processor location?

At distance 3, the logical-error rate of each decoder varies across the 4 processor locations (belief_matching 0.388–0.416, correlated_matching 0.413–0.425, pymatching 0.430–0.443, tensor_network_contraction 0.384–0.413). The decoder with the lowest rate differs by location:

1. centre 3_5: tensor_network_contraction
2. centre 5_3: tensor_network_contraction

3. centre 5_7: tensor_network_contraction

4. centre 7_5: belief_matching

At distance 5 ,one experiment at centre 5_5, the rates range from 0.396 for tensor_network_contraction to 0.451 for pymatching, similar to distance 3. The actual flip rate is between 0.490 and 0.500 everywhere: over 25 rounds the observable flips in about half of the shots, so all decoders stay near 0.4. With one distance-5 experiment, distance and location cannot be separated. The differences are associations between experiments, not effects of location or distance. This query joins four Gold tables: google_experiment, google_shot, decoder_prediction and decoder.

#### 6.2.3 How does the repetition-code circuit map data qubits to parity-check ancillas, syndrome bits, and conditional corrections?

In qec_sm_n5, a[0] measures the parity of q[0], q[1] into syn[0], a[1] measures the parity of q[1], q[2] into syn[1]. A correction is conditioned on the whole two-bit syndrome register, and bit k of its value is the syndrome bit written by check k: syn = 1: a[0] fires $\rightarrow$ x q[0]; syn = 2: a[1] fires $\rightarrow$ x q[2]; syn = 3: a[0] and a[1] fire $\rightarrow$ x q[1]. In every case the corrected qubit is the only data qubit that belongs to all fired checks and to no silent check — the flipped qubit the syndrome points to. The transpiled variant has the same mapping.

# 7 ML input tables

Part II reads only two Parquet tables whose columns, types and meaning are fixed by the course contract: `ml_syndrome_decoder_example` and `ml_google_decoder_example`. Stage 4 of `make run` (`build_ml_tables`) builds both from PostgreSQL Gold only; it never re-reads Bronze or Silver.

## 7.1 Gold views

The content of both tables is defined in committed SQL views:

**Stable example identifiers.** `example_id` is a SHA-256 hash of the business key, so it is identical on every run and independent of load order. For syndromes the key is experiment, syndrome bits *and* label, because the same syndrome legitimately occurs with both labels (Section 4). For Google shots the key is experiment plus shot index.

## 7.2 Export step

A thin Python step creates the views, queries them in a fixed order, and adds `data_split` with the supplied helpers: by physical fault rate for syndromes and by shot index for Google shots. It then checks the contract and writes both tables as Parquet to MinIO (`ml/`). The column order and types are enforced by an explicit Arrow schema, so an extra, missing or wrongly typed column fails the export. The Gold `source_record_id` is used for lineage inside the views but is not exported.

Table 6: Description of database views and corresponding SQL scripts.

| View (file) | What it does |
|---|---|
| v_ml_syndrome_decoder_example (ml_syndrome_example.sql) | Joins syndrome_observation, sim_experiment, and syndrome_pattern. One row per syndrome observation; the fault rate comes from the experiment, the round and check counts from the pattern, and sample_weight is the observation's quantity. |
| v_ml_google_decoder_example (ml_google_example.sql) | Joins google_shot, google_experiment, and decoder_prediction, and pivots the four predictions of a shot into four columns. One row per shot. |
| v_ml_example_gold_record (ml_example_gold_record.sql) | Maps every example_id to the Gold records it was built from: one syndrome_observation for a syndrome example; one google_shot and four decoder_prediction rows for a Google example. |

Before writing, the export stops the run if any of these checks fails:

- example_id is unique, and all three splits are present and non-empty, with no other split values;
- syndromes: 4 rounds $\times$ 4 checks, 16 binary values in round order (supplied helper), and a positive sample_weight;
- Google: 200 detectors for distance 3 and 600 for distance 5; the packed detector bits have $\lceil n/8 \rceil$ bytes with zero padding; the little-endian unpacked bits sum to detector_event_count; and the label and all four predictions are present, i.e. aligned to the shot;
- every example resolves to exactly its expected Gold records (1 for a syndrome, 5 for a Google shot) through v_ml_example_gold_record.

The course data-release identifier is stored in each file's Parquet metadata rather than as a column, so the contract columns stay exact while Part II can record which release it trained on. The row counts, split counts and SHA-256 hash of each file are written to run.json.

Table 7: Dataset row splits across training, validation, and test sets.

| Table | Rows | Train | Validation | Test |
|---|---|---|---|---|
| ml_syndrome_decoder_example | 75 598 | 53 304 | 1 407 | 20 887 |
| ml_google_decoder_example | 250 000 | 100 000 | 25 000 | 125 000 |

The syndrome splits are whole experiments: validation is the fault-rate $5 \cdot 10^{-4}$ file and test is the $5 \cdot 10^{-3}$ file, so no fault rate is seen in two splits. The split sizes in rows are therefore very uneven, but the splits are used exactly as supplied. Two

consecutive runs produce byte-identical Parquet files (equal hashes); 11 tests cover the contract checks, the schema and the resolution of example identifiers.

**Design choice.** The ML content lives in views rather than materialised tables, so it can never drift from Gold. The cost is that the joins are recomputed on every export (about 30 seconds).

# 8 Tracing

The tracing stage of the pipeline takes a google_prediction and a syndrome_observation and traces it back from the row prediction throughout all the stages from the prediction, the ML table, to Gold, Silver, the source parquet and at last the Bronze stage.

## 8.1 Google_prediction

1. Prediction : for our example_id, we get predicted value true with probability 0.555 against the ground truth label true in the test split

2. ML row : the example_id resolves to one row in the ml_google_decoder_example.parquet experiment_id = surface_code_bX_d3_r25_center_3_5, shot_index = 1, distance 3, 25 rounds, processor location (center_row=3, center_col=5)

3. Gold : the v_ml_example_gold_record resolves the same example_id to 5 Gold rows sharing the same source_record_id : 1 google_shot and 4 decoder_prediction

4. Silver : the source_record_id is one row in the google_qec/shot.parquet and it contains the same bytes and all 4 decoder predictions, therefore Gold did not alter anything in between stages

5. source_trace.parquet : one shot is assembled from multiple aligned companion files, the source_record_id is mapped to 8 trace rows: measurements.b8, sweep.b8, detection_events.b8, obs_flips_actual.01, and the four obs_flips_predicted_by_*.01 files — each recording its own archive_member and record_locator

6. Bronze : all 8 traces point to the same bronze object, bronze/source=google_qec/google-surface-code-curated.zip, and all have the same SHA-256, which is also verified since the matches_verified_input_hash is true

## 8.2 Syndrome_observation

1. Prediction : for our example_id we get predicted value false with probability 0.0174 against the ground truth label false in the test split

2. ML row : the example_id resolves to one row in the ml_syndrome_decoder_example.parquet a 16-byte all-zero syndrome_bits pattern (4 rounds × 4 checks, no triggered checks), sample_weight = 5,309,458

3. Gold : resolves to exactly 1 syndrome_observation row

4. Silver : same source_record_id one row of silver/qec_syndromes/syndrome_observation.parquet, matching the Gold's value exactly

5. source_trace.parquet : just 1 trace, the syndrome observation comes from a single csv row

6. Bronze : the trace points to bronze/source=qec_syndromes/syndromes_dataset.zip, and the SHA-256 is verified as well

# 9 Reproducibility

Reproducibility is supported through deterministic record identifiers, verified input files, and repeatable pipeline stages. Before processing, the pipeline checks the size and SHA-256 hash of each supplied Bronze object against the release manifest. Record identifiers are derived from the source, archive member, and record position rather than from the execution time or load order. Consequently, unchanged inputs produce stable identifiers across runs.

Repeatability is tested at both the Silver and Gold stages. The Silver test runs the preparation stage twice and compares the row counts and sorted `source_record_id` values of all six Parquet tables. The Gold integration test similarly compares the keys and row counts of all twelve PostgreSQL tables after two consecutive loads, and checks for duplicate keys. Additional assertions verify that the loaded Gold row counts agree with their corresponding Silver tables.

For the ML hand-off, the two input tables are generated from committed SQL views over Gold, with deterministic example identifiers and data-split assignments. Contract checks validate the table schemas, unique example identifiers, non-empty splits, and the alignment of Google detector bits, logical outcomes, and decoder predictions. SHA-256 hashes and row counts are recorded for the generated tables to support subsequent verification.

Together, these mechanisms make changes to inputs, identifiers, row counts, or exported data detectable and allow the pipeline outputs to be checked across repeated executions.